

EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

Program Studi Teknik Instrumentasi
Semester Genap 2025/2026

PROJECT BASED LEARNING (PBL)

**Tank Level Monitoring System untuk Manajemen Air
Gedung
Berbasis Rust**

Dosen Pengampu : Ahmad Radhy, S.Si., M.Si.
Durasi Pengerjaan : 1 Pekan
Bahasa Pemrograman : Rust
Metode Penilaian : Project + Ujian Lisan

Kelompok :

Mahasiswa 1 : Mufflih Azhar
NRP : 2042251079

Mahasiswa 2 : M. Nafis Abdul Majid
NRP : 2042251126

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember

Daftar Isi

1	Pendahuluan	2
2	Studi Kasus Instrumentasi	2
3	Algoritma dan Flowchart	2
4	Konsep Pemrograman Berbasis Objek (OOP)	3
5	Implementasi Komputasi Numerik	3
6	Penjelasan Program Rust	4
7	Hasil Pengujian	6
8	Analisis Hasil	7
9	Kesimpulan	7

1 Pendahuluan

Sistem penyediaan air pada gedung-gedung besar sering kali menghadapi kendala dalam akurasi pemantauan level air, yang jika dilakukan secara manual dapat menyebabkan pemborosan energi atau keterlambatan pengisian. Proyek ini merancang sistem monitoring level tangki otomatis berbasis terminal menggunakan bahasa pemrograman Rust. Fokus utama sistem ini adalah menjaga kestabilan data melalui filter digital dan memastikan akurasi melalui tahap kalibrasi mandiri sebelum mengaktifkan pompa.

2 Studi Kasus Instrumentasi

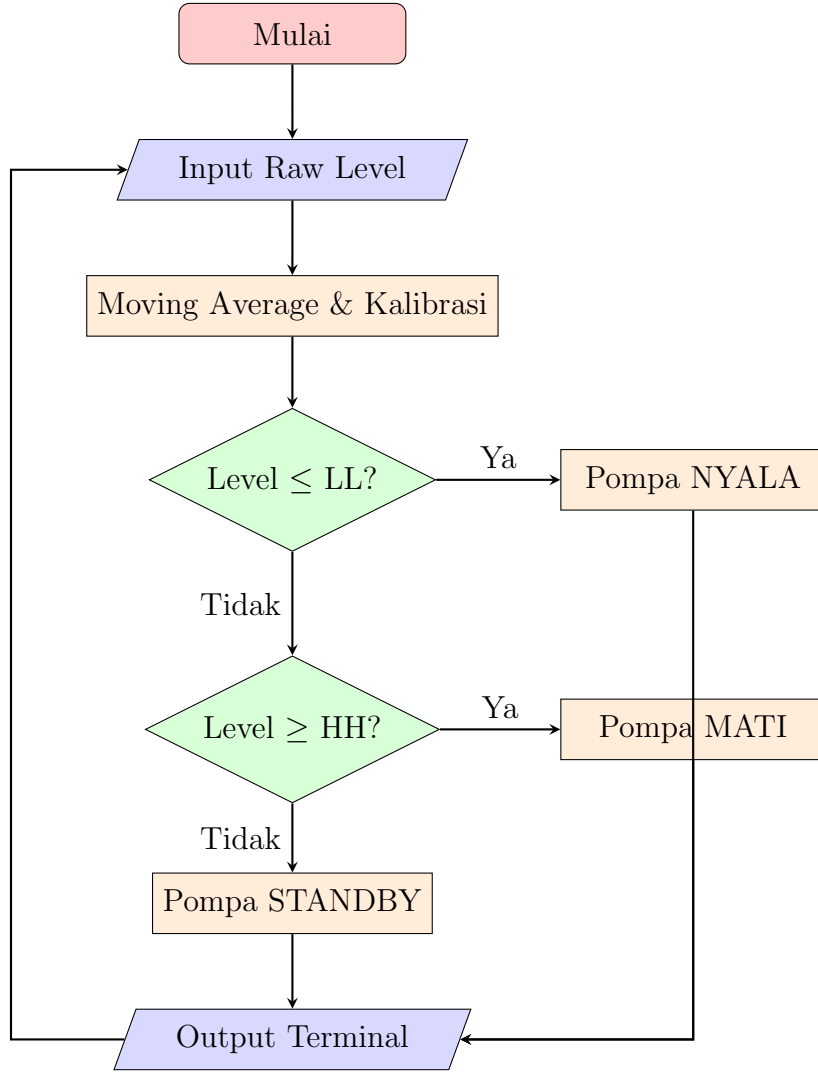
Proyek ini mengambil studi kasus sistem monitoring tandon air dengan parameter desain sebagai berikut:

- **Objek Ukur:** Tangki silinder dengan jari-jari (r) sebesar 2.0 meter.
- **Batas Ambang:** Batas Bawah (*Low Level*) ditetapkan pada 2.0 meter untuk memicu pompa nyala, dan Batas Atas (*High-High*) pada 9.0 meter untuk mematikan pompa.
- **Kompensasi Galat:** Sensor diasumsikan memiliki galat sistematis sebesar 0.5 meter yang dikoreksi melalui perangkat lunak.

3 Algoritma dan Flowchart

Algoritma sistem berjalan secara interaktif sebagai berikut[:

1. Menerima input pembacaan sensor mentah (*raw data*) dari pengguna melalui terminal.
2. Melakukan perhitungan *Simple Moving Average* (SMA) menggunakan 3 data terakhir untuk mereduksi noise.
3. Melakukan kalibrasi dengan menjumlahkan hasil rata-rata dengan nilai galat (0.5m) untuk mendapatkan ketinggian final.
4. Menghitung volume air aktual menggunakan rumus geometri silinder.
5. Melakukan pengambilan keputusan (*decision making*) status pompa berdasarkan ambang batas LL dan HH.



Gambar 1: Alur Logika Sistem Kendali Level Tangki Interaktif

4 Konsep Pemrograman Berbasis Objek (OOP)

Implementasi OOP pada program ini menggunakan **struct TankSystem**. Seluruh variabel penting (kapasitas, tinggi, error) dibungkus (enkapsulasi) dalam satu struktur data. Fungsi-fungsi terkait operasi tangki dijadikan sebagai **method** dalam blok **impl**, sehingga memudahkan pemanggilan fungsi yang terpusat pada objek tangki tersebut.

5 Implementasi Komputasi Numerik

Dua metode numerik utama diterapkan dalam sistem ini:

1. **Moving Average:** Menghitung rata-rata dari $n = 3$ data terakhir untuk stabilitas pembacaan.

$$h_{avg} = \frac{h_i + h_{i-1} + h_{i-2}}{3} \quad (1)$$

2. **Kalibrasi Mandiri:** Menyesuaikan nilai rata-rata dengan galat sistematis.

$$h_{final} = h_{avg} + \text{error} \quad (2)$$

3. **Perhitungan Geometri Silinder:** Menghitung volume air aktual (V) di dalam tandon berbentuk silinder berdasarkan nilai ketinggian yang telah diproses (h_{final}) dengan parameter jari-jari tangki (r) sebesar 2.0 meter.

$$V = \pi \cdot r^2 \cdot h_{final} \quad (3)$$

6 Penjelasan Program Rust

Kode berikut dirancang untuk berjalan di terminal secara interaktif:

```
1 use std::io::{self, Write};
2
3 struct Sensor {
4     name: String,
5     readings: Vec<f32>,
6     window_size: usize,
7     calibration_error: f32,
8 }
9
10 impl Sensor {
11     fn new(name: &str, window_size: usize, error: f32) ->
12     Self {
13         Self {
14             name: name.to_string(),
15             readings: Vec::new(),
16             window_size,
17             calibration_error: error,
18         }
19     }
20
21     fn moving_average(&mut self, new_value: f32) -> f32 {
22         self.readings.push(new_value);
23         if self.readings.len() > self.window_size {
24             self.readings.remove(0);
25         }
26         let sum: f32 = self.readings.iter().sum();
27         sum / self.readings.len() as f32
28     }
29
30     fn calibrate(&self, value: f32) -> f32 {
31         value + self.calibration_error
32     }
33
34     fn get_processed_level(&mut self, raw_value: f32) -> f32
35     {
```

```

34         let averaged = self.moving_average(raw_value);
35         self.calibrate(averaged)
36     }
37 }
38
39 struct MonitoringSystem {
40     sensor: Sensor,
41     low_limit: f32,
42     high_limit: f32,
43 }
44
45 impl MonitoringSystem {
46     fn new(ll: f32, hh: f32, err: f32) -> Self {
47         Self {
48             sensor: Sensor::new("Water Level", 3, err),
49             low_limit: ll,
50             high_limit: hh,
51         }
52     }
53
54     fn calculate_volume(&self, height: f32) -> f32 {
55         let radius: f32 = 2.0;
56         let pi = 3.14159;
57         pi * radius * radius * height
58     }
59
60     fn run_process(&mut self, raw_val: f32) {
61         let processed = self.sensor.get_processed_level(
raw_val);
62         let volume = self.calculate_volume(processed);
63
64         let status_pompa = if processed <= self.low_limit {
65             "NYALA [>>>] - Mengisi air"
66         } else if processed >= self.high_limit {
67             "MATI [____] - Tangki penuh"
68         } else {
69             "STANDBY [----]"
70         };
71
72         println!("
-----");
73         println!("DATA SENSOR | Raw: {:.2}m | Final: {:.2}m"
, raw_val, processed);
74         println!("VOLUME AIR | {:.2} m3", volume);
75         println!("STATUS POMPA | {}", status_pompa);
76     }
77 }
78
79 fn main() {

```

```

80     let mut system = MonitoringSystem::new(2.0, 9.0, 0.5);
81
82     println!("=== SISTEM MONITORING TANGKI INTERAKTIF ===");
83     println!("Batas Bawah: {}m | Batas Atas: {}m", system.
low_limit, system.high_limit);
84     println!("Ketik 'exit' untuk berhenti.\n");
85
86     loop {
87         print!("Masukkan pembacaan sensor (meter): ");
88         io::stdout().flush().unwrap();
89
90         let mut input_text = String::new();
91         io::stdin()
92             .read_line(&mut input_text)
93             .expect("Gagal membaca input");
94
95         let trimmed = input_text.trim();
96         if trimmed == "exit" {
97             println!("Program berhenti. Sampai jumpa!");
98             break;
99         }
100
101         match trimmed.parse::<f32>() {
102             Ok(val) => {
103                 system.run_process(val);
104                 println!("
-----\n");
105             }
106             Err(_) => {
107                 println!("Input tidak valid! Harap masukkan
angka.\n");
108             }
109         };
110     }
111 }

```

Listing 1: Implementasi Sistem Monitoring Sensor dan Kendali Pompa Interaktif dengan Rust

7 Hasil Pengujian

Pengujian dilakukan dengan simulasi data input melalui terminal secara berurutan:

No	Input Average	Ketinggian Final	Volume (m^3)	Status Pompa
1	1.50 m	2.00 m	25.13	NYALA (LL Tercapai)
2	5.00 m	5.50 m	69.12	STANDBY
3	8.50 m	9.00 m	113.10	MATI (HH Tercapai)

8 Analisis Hasil

Berdasarkan hasil pengujian, sistem menunjukkan stabilitas yang baik berkat penggunaan jendela rata-rata 3 data. Kalibrasi penambahan 0.5 meter efektif dalam mengimbangi deviasi sensor sehingga pompa aktif tepat pada batas fisik LL 2.0 meter. Perhitungan volume silinder memberikan informasi tambahan yang krusial bagi operator untuk memantau kapasitas tersedia secara presisi

9 Kesimpulan

Sistem monitoring level tangki interaktif ini berhasil memenuhi standar evaluasi dengan mengintegrasikan konsep OOP, algoritma kendali, dan komputasi numerik dalam bahasa Rust. Penggunaan filter *Moving Average* dan kalibrasi mandiri terbukti meningkatkan keandalan sistem dalam mendeteksi batas kritis LL dan HH secara akurat.